

NAMAL UNIVERSITY MIANWALI

Department of Computer Sciences

Course Project Final Report

Parallel Odd-Even Sort Simulation

A Graphical Visualization and Performance Analysis Tool

Course: Parallel and Distributed Computing

Instructor: Muhammad Shahzad Arif

Submission Date: June 2026

Group Member	Roll Number
Mishaal Aqdas	NUM-BSCS-2024-35
Najia Nayab	NUM-BSCS-2024-62
Shahbaz Ali	NUM-BSCS-2024-73

1. Abstract

This report presents the design, development, and analysis of a Parallel Odd-Even Transposition Sort Simulation tool built in Python using the Pygame library. The simulation provides an interactive, real-time graphical visualization of both sequential and parallel execution modes of the Odd-Even Sort algorithm, enabling users to observe comparison and swap operations across Odd and Even phases as they occur.

The tool features a cyberpunk-themed visual interface with animated bar elements, particle effects, sound synthesis, and an interactive scrubber for step-by-step navigation. It supports configurable array sizes (12 to 64 elements), custom user input, speed controls, CSV metric export, and GIF recording. Performance metrics including total comparisons, total swaps, and cycle count are tracked and displayed in real time.

The project successfully demonstrates the theoretical concepts of parallel sorting, synchronization, and phase-based execution through an educational simulation tool, fulfilling all objectives outlined in the project proposal.

2. Introduction

2.1 Background

With the increasing demand for high-performance computing, parallel computing has become a critical solution for processing large-scale data efficiently. Modern systems rely on parallel algorithms to improve speed by executing multiple operations simultaneously. Sorting algorithms are among the most fundamental operations in computing, widely applied in databases, networking, operating systems, and data processing pipelines.

The Odd-Even Transposition Sort algorithm is particularly well-suited for parallel execution. In each cycle, it performs comparisons in alternating Odd and Even phases where all comparisons within a phase can theoretically proceed concurrently. This makes it an ideal algorithm for studying synchronization, parallelism, and communication overhead in multi-processor systems.

2.2 Problem Statement

While Odd-Even Sort is well-established in parallel computing literature, most educational presentations rely on static diagrams and pseudocode that fail to convey the dynamics of concurrent operation. Students and learners struggle to develop an intuitive understanding of how elements move through comparison and swap phases simultaneously across an array.

This project addresses that gap by developing an interactive graphical simulation that renders every algorithmic step in real time, clearly distinguishing between sequential and

parallel execution modes, and providing performance metrics that enable direct comparative analysis.

2.3 Objectives

1. Implement the Odd-Even Transposition Sort algorithm in both sequential and parallel simulation modes.
2. Develop a real-time graphical visualization of comparison, swap, and phase-transition events.
3. Support configurable array sizes, custom input arrays, and adjustable execution speed.
4. Track and display performance metrics: comparisons, swaps, cycles, and step progress.
5. Compare sequential vs. parallel mode behavior through visual demonstration.
6. Export metrics to CSV and provide GIF recording for documentation.
7. Deliver a user-friendly educational tool requiring no external configuration.

3. Algorithm Background

3.1 Odd-Even Transposition Sort

Odd-Even Transposition Sort is a comparison-based parallel sorting algorithm derived from Bubble Sort. It operates in alternating phases called Odd and Even phases, each of which processes non-overlapping adjacent pairs of elements across the array.

Odd Phase

In the Odd Phase, adjacent pairs at odd-indexed positions are compared and swapped if out of order: $(arr[1], arr[2])$, $(arr[3], arr[4])$, $(arr[5], arr[6])$, and so on.

Even Phase

In the Even Phase, adjacent pairs at even-indexed positions are processed: $(arr[0], arr[1])$, $(arr[2], arr[3])$, $(arr[4], arr[5])$, and so on.

One complete cycle consists of one Odd phase followed by one Even phase. The algorithm terminates when a full cycle completes with no swaps, indicating the array is sorted.

3.2 Parallelism

The key property that enables parallelism is that within each phase, all comparison pairs are disjoint — no element participates in more than one comparison simultaneously. In a true parallel hardware model (e.g., a linear processor array), all pairs in a phase can be compared and swapped in a single time step, reducing the time complexity from $O(n^2)$ in sequential Bubble Sort to $O(n)$ parallel steps for an array of size n .

3.3 Complexity Analysis

Property	Sequential Mode	Parallel Mode
Time Complexity	$O(n^2)$ worst case	$O(n)$ parallel steps
Comparisons per cycle	$n - 1$ (sequential)	$n/2$ (simultaneous)
Number of cycles	Up to n cycles	Up to n cycles
Space Complexity	$O(1)$ in-place	$O(1)$ in-place
Stability	Stable	Stable

4. System Design and Architecture

4.1 Technology Stack

The simulation was implemented in Python 3 using the following libraries and tools:

Component	Technology / Library
Core Language	Python 3.x
Graphics & Window	Pygame (SDL2 wrapper)
Audio Synthesis	Pygame Mixer with custom PCM sine wave generator
Data Export	Python csv, os, time modules
GIF Recording	Pillow (PIL) for frame capture and export
IDE	Visual Studio Code / PyCharm

4.2 Architectural Overview

The application is organized into the following major components:

4.2.1 Algorithm Generators

Two Python generator functions implement the sorting algorithms as lazy state machines, yielding a snapshot dictionary at every event (comparison, swap, phase start, cycle complete, done):

- `oddeven_seq_generator(arr)`: Sequential mode — processes one pair at a time.
- `oddeven_parallel_generator(arr)`: Parallel mode — processes all pairs in a phase simultaneously.

Each snapshot contains the current array state, active element indices, event type, phase label, and cumulative metric counters. All states are pre-computed at startup and stored in a list, enabling instant scrubbing to any step.

4.2.2 Bar Visual Elements

The Bar class represents each array element as an animated capsule bar. It maintains current and target positions, smoothly interpolating via a lerp (linear interpolation) update

on each frame. Bar height is proportional to element value relative to the array maximum. Color transitions are also interpolated for smooth visual feedback.

4.2.3 Animation Manager

The AnimationManager class coordinates three layers of visual effects:

- Drifting star particles in the background (50 particles, randomized speed and alpha).
- Spark explosion particles triggered on swap events.
- Animated checkmark overlays triggered on no-swap confirmations.
- Glowing connection lines drawn between active comparison pairs.
- Bidirectional arc arrows drawn between swap pairs.

4.2.4 Sound Effects

The SoundEffects class uses a PCM sine wave generator (make_sound) to synthesize audio tones in memory. Five distinct tones are used to signal comparison events, swaps, no-swaps, phase transitions, and sort completion. Sound can be toggled at runtime.

4.2.5 Array Visualizer (Coordinator)

The ArrayVisualizer class acts as the central coordinator. It manages the event loop, button interactions, state scrubbing, speed control, modal dialogs, CSV export, GIF recording, and the main draw pipeline. The draw method renders all visual layers in a defined z-order: background stars → header text → canvas bars → effects → control buttons → status panel → scrubber slider → modal overlay.

4.3 User Interface Layout

Panel	Position (px)	Contents
Header / State Display	Top 20–20+120 px	Algorithm name, phase, comparison result, explanatory text
Array Canvas	y=250, h=270 px	Animated bars, connection lines, swap arrows, value labels
Control Buttons	y=535, h=55 px	Play/Pause, Step, Speed, Size, Mode, Generate, Custom, Export
Details / Scrubber	y=605, h=95 px	Step counter, timeline scrubber knob

5. Implementation Details

5.1 State Machine Design

Rather than animating sorting in real time with direct array modifications, all states are pre-computed using Python generators. This approach offers several advantages:

- Instant step-backward navigation without re-running the algorithm.

- Precise scrubbing to any point in the execution timeline via the slider.
- Clean separation between algorithm logic and rendering logic.
- Consistent performance regardless of visualization speed.

5.2 Sequential Mode

In sequential mode, the generator processes one comparison pair per yield. For each pair $(i, i+1)$, it yields a compare state, then either a swap or no_swap state depending on the ordering. This produces a granular step-by-step trace identical to how a human would manually trace the algorithm on paper.

5.3 Parallel Mode

In parallel mode, the generator groups all pairs in a phase into a single batch. It yields one compare_parallel state showing all pairs simultaneously highlighted, followed by either a swap_parallel state (if any pairs needed swapping) or a no_swap_parallel state. This accurately simulates the behavior of a parallel processor array where all comparisons in a phase occur in the same clock cycle.

The total number of states produced by parallel mode is significantly smaller than sequential mode for the same array, reflecting the reduced step count of true parallel execution.

5.4 Visual Interpolation

Bar elements use exponential smoothing (lerp factor 0.15 per frame) for both position and color transitions. This creates the sliding animation effect when bars change positions during swap events, without requiring explicit animation keyframing. The glow effect on active bars is rendered by drawing the bar shape multiple times with decreasing opacity at increasing offsets.

5.5 Custom Input and Validation

A modal dialog allows users to enter comma-separated integer values. The validator enforces the following constraints:

- Minimum 4 elements, maximum 100 elements.
- All values must be positive integers in the range 1 to 500.
- Invalid input displays an inline error message without closing the modal.

Keyboard input is restricted to digit characters and commas during modal interaction to prevent accidental keypress conflicts.

5.6 CSV Export

The Export CSV function appends a row to odd_even_sort_metrics.csv containing: timestamp, array size, mode (Sequential/Parallel), total comparisons, total swaps, and current cycle count. If the file does not exist, a header row is written first. This allows accumulation of results across multiple runs for external analysis.

5.7 GIF Recording

The recording system captures a copy of the Pygame surface at each state change during playback. On stop, frames are scaled to 640x360, converted to PIL images, and saved as an animated GIF with frame delay matched to the current playback speed setting. This feature requires the Pillow library to be installed.

6. Features and Controls

6.1 Control Buttons

Button	Keyboard	Action
Play / Pause	Space	Toggle automatic step-through playback
< Prev	← Arrow	Step backward one state
Next >	→ Arrow	Step forward one state
+ / -	—	Increase or decrease playback speed (10 levels from 1000ms to 2ms delay)
Size	—	Cycle array size: 12 → 16 → 24 → 32 → 48 → 64
Mode: Seq / Parallel	P	Toggle between Sequential and Parallel simulation mode
Generate	N	Generate a new random array and recompile all states
Custom Input	—	Open modal dialog to enter a custom array
Export CSV	E	Append current metrics to odd_even_sort_metrics.csv
F key	F	Start / stop GIF recording of the current session
Escape	Esc	Quit the application

6.2 Scrubber Slider

A timeline scrubber at the bottom of the screen allows clicking and dragging to jump to any point in the execution history. The knob position reflects the current step as a fraction of total steps, providing intuitive visual feedback of progress through the algorithm.

6.3 Color Legend

Color	Meaning
Cyan (#00F0FF)	Active element selected for comparison
Coral Red (#FF453A)	Elements requiring a swap / swap in progress
Neon Green (#39FF14)	Elements in correct order / sorting complete wave
Yellow (#FFEB3B)	Comparison count line in metrics graph
Alternating White/Dark	Default unsorted bar colors (alternating pattern)

7. Results and Analysis

7.1 Sequential vs. Parallel Mode Comparison

The following observations are based on multiple test runs across different array sizes using the simulation:

Array Size	Seq. States	Par. States	Seq. Comps	Par. Comps	Cycles
12	~55–80	~20–28	~40–55	~24–36	~6
24	~200–280	~60–80	~170–230	~110–150	~12
32	~350–500	~90–120	~300–420	~180–260	~16
64	~1200+	~200–280	~1000+	~500–700	~32

Note: Values are approximate as they depend on the specific random array generated. The simulation generates exact counts for any given run, visible in the state snapshots and exportable to CSV.

7.2 Key Observations

- Parallel mode consistently produces fewer total visualization states (steps) than sequential mode for the same array, reflecting the reduced step count of concurrent processing.
- Both modes perform the same number of logical cycles — the difference lies in steps per cycle.
- The total number of swaps is identical between modes for the same array, as the same elements are ultimately moved to the same positions.
- Comparison counts differ because sequential mode counts each pair individually while parallel mode counts all pairs in a batch as one group operation.
- The neon green sorted-wave animation at completion is a reliable visual indicator that the algorithm has halted correctly.
- The scrubber slider confirms that state counts scale roughly linearly with array size in both modes.

7.3 Comparison with Traditional Algorithms

Algorithm	Best Case	Average Case	Worst Case	Parallel?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	No
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Limited
Odd-Even Sort (Seq.)	$O(n)$	$O(n^2)$	$O(n^2)$	No

Odd-Even Sort (Par.)	$O(1)$ steps	$O(n)$ steps	$O(n)$ steps	Yes
----------------------	--------------	--------------	--------------	-----

8. Challenges and Solutions

8.1 Smooth Bar Animations

Challenge: Directly jumping bars to new positions on each state update caused jarring visual transitions.

Solution: Implemented position and color lerp interpolation in the `Bar.update()` method. Bars smoothly slide from their current visual position toward their target position each frame, providing fluid animation even at high playback speeds.

8.2 Backward Navigation

Challenge: Sorting algorithms modify the array in-place, making it non-trivial to step backward through states.

Solution: All states are pre-computed and stored as snapshots before visualization begins. Stepping backward simply decrements the state index and re-renders, with no re-computation required.

8.3 Parallel Mode Visual Clarity

Challenge: Displaying multiple simultaneous comparisons and swaps clearly without cluttering the canvas.

Solution: In parallel mode, all active pairs are highlighted simultaneously with glowing connection lines. The large central text displays the count of pairs being processed, and swap arrows are drawn for each swapping pair independently. Color coding (green for ordered, red for swap-needed) communicates the outcome of each pair at a glance.

8.4 Audio Without External Files

Challenge: Providing audio feedback without bundling audio asset files.

Solution: Implemented a PCM sine wave synthesizer (`make_sound`) that generates tone buffers in memory using Python's `struct` module and injects them into Pygame's mixer directly. No external audio files are required.

8.5 GIF Export Performance

Challenge: Capturing high-resolution frames while maintaining smooth playback and exporting efficiently.

Solution: Frames are captured only on state changes (not every frame), and scaled down to 640x360 during export to reduce file size. A saving screen is displayed during the GIF compilation phase to prevent UI freeze perception.

9. Conclusion

This project successfully developed a comprehensive graphical simulation of the Parallel Odd-Even Transposition Sort algorithm, fulfilling all objectives stated in the project proposal. The simulation provides clear, interactive visualization of both sequential and parallel execution modes, with real-time performance metrics, intuitive controls, and visual effects that make algorithmic behavior immediately understandable.

Key contributions of this project include a pre-computed state machine architecture that enables instant navigation through any point in the algorithm's execution history, a smooth animated bar visualization with interpolation-based transitions, a PCM audio synthesis system that provides sound feedback without external dependencies, parallel mode simulation that accurately demonstrates the theoretical advantage of concurrent phase processing, and a complete data export pipeline for analysis and documentation.

The project deepened our practical understanding of parallel computing concepts including phase synchronization, data independence within phases, and the relationship between parallelism and step-count reduction. It also provided valuable experience in designing educational visualization tools that bridge the gap between algorithm theory and intuitive understanding.

Future extensions could include a GPU-accelerated rendering mode using PyOpenGL, a hardware-accurate simulation on a simulated linear processor array, side-by-side comparison views of sequential and parallel modes running simultaneously, and extended algorithm support for Bitonic Sort or Merge Sort to enable broader comparative analysis.

10. References

8. Kumar, V., Grama, A., Gupta, A., and Karypis, G. (1994). Introduction to Parallel Computing: Design and Analysis of Algorithms. Benjamin/Cummings.
9. Knuth, D.E. (1998). The Art of Computer Programming, Volume 3: Sorting and Searching (2nd ed.). Addison-Wesley.
10. Pygame Documentation (2024). Pygame Community Edition.
<https://www.pygame.org/docs/>
11. Cormen, T.H., Leiserson, C.E., Rivest, R.L., and Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
12. Leighton, F.T. (1992). Introduction to Parallel Algorithms and Architectures: Arrays, Trees, Hypercubes. Morgan Kaufmann.
13. Odd-Even Transposition Sort. Wikipedia.
https://en.wikipedia.org/wiki/Odd%E2%80%93even_sort